



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

November 30th, 2022

December 2nd, 2022

FILES

- Files are named locations on disk to store related information
- They are used to permanently store data in a non-volatile memory (e.g. hard disk, flash drive, etc)
- Files can be of different types:
 - Data file (Word processing, Text, Images, Database, etc)
 - Code file (Programming language scripts)
 - Executable file (Executable software, desktop applications, mobile apps, etc)

BASIC FILE OPERATIONS

- When we want to read from or write to a file, we need to open it first
- When we are done, it needs to be closed so that the resources that are tied with the file are freed
- Hence, in Python, a file operation takes place in the following order:
 - Open a file
 - Read or write (perform operation)
 - Close the file

OPENING A FILE

- Python has a built-in `open()` function to open a file
- This function returns a file object, also called a ***handle***, as it is used to read or modify the file accordingly
- Example:
 - `f = open("test.txt")` # open file in current directory
 - `f = open("C:/Python38/README.txt")` # specifying full path
- By default, if just the file name is specified in the argument, it is assumed that it is being opened for reading purposes only
- What if we want to open it for writing?

OPENING A FILE

- The built-in function, *open()* is overloaded with additional arguments to specify the purpose of opening a file
- This is called the ***mode*** of the file
- We can specify the mode while opening a file
- In mode, we specify whether we want to
 - Read “r” from a file
 - Write “w” to a file
 - Append “a” to a file
- Example
 - `f = open("test.txt")` # equivalent to 'r'
 - `f = open("test.txt", mode = 'w')` # write in text mode

OPENING A FILE

- Modes can also be used to specify what kind of file are we trying to open
- Must always be specified with read/write/append/update modes
- It has two options:
 - Text mode that reads data as strings
 - Default mode
 - Used mostly for data files
 - Binary mode that returns data in bytes
 - Must be explicitly specified, otherwise text mode is assumed irrespective of the file type
 - Mostly used when dealing with non-text files, like images or executable files)
- Example:
 - `f = open("test.txt", mode = 'w')` `# write in text mode`
 - `f = open("img.bmp", mode = 'rb')` `# read in binary mode`

CLOSING A FILE

- When we are done with performing operations on the file, we need to properly close the file
- Closing a file will free up the resources that were tied with the file
- It is done using the *close()* method available in Python
- Python has a garbage collector to clean up unreferenced objects but we must not rely on it to close the file
- Example:
 - `f = open("test.txt")`
 - `# perform file operations`
 - `f.close()`

READING A FILE

- To read a file in Python, we must open the file in reading “r” mode
- The various built-in methods used to read from a file are:
 - ***read()*** – reads the file from the current position till the end
 - ***read(size)*** – reads the “size” number of characters from the current location
 - ***readline()*** – reads a file from the current location till the newline, including the newline character
 - ***readlines()*** – reads the remaining lines of the entire file (Similar to ***read()*** method)
- All these reading methods return empty values when the end of file (EOF) is reached
- Each of these methods returns a newline as '\n'

READING A FILE

- Example:
 - Assume test.txt contains the following text:
 - This is my first file
This file
contains three lines
 - `f = open("test.txt", 'r')`
 - `X = f.read(4)` # read the first 4 data
`print(X)`
 - Output → "This"
 - `print(f.read(4))` # read the next 4 data
 - Output → " is "
 - `print(f.read())` # read in the rest till end of file
 - Output → 'my first file\nThis file\ncontains three lines\n'
 - `print(f.read())` # further reading returns empty sting
 - Output → ""
 - `f.close()`

READING A FILE

- If you want to find out and/or update from the current cursor position in the file, you may use the following the following methods:
 - ***tell()*** – returns the current cursor position (in number of bytes)
 - ***seek(position)*** – takes the cursor to the “position” (in number of bytes) in the file, where each character is 1 byte long

READING A FILE

- Example:
 - Assume test.txt contains the following text:
 - This is my first file
This file
contains three lines
 - `f = open("test.txt", 'r')`
 - `X = f.read(4)` # read the first 4 data
`print(X)`
 - Output → "This"
 - `print(f.tell())`
 - Output → 5
 - `f.seek(1)`
 - `print(f.readline())`
 - Output → "his is my first file"

CLOSING A FILE

- If you forget to close the file, and run the program again, you may end up getting Access Denied error
- To avoid this, use a filing keyword “with” that ensures that once the module completes the task, it closes the file for you automatically
- Example:
 - with open("test.txt") as f:
 #perform file operations

WRITING TO A FILE

- In order to write into a file in Python, we need to open it in write 'w', or append 'a'
- In each of these modes, if the file does not exist, it creates a new one with the same name
- For 'w' mode, it completely overrides an existing file, removing all the data from before
- For 'a' mode, it adds to the file from the current cursor position

WRITING TO A FILE

- Example:
 - `f = open("test.txt", 'w')`
`f.write("Adding new!")`
`f.close()`
 - `with open("test.txt", 'a') as f:`
`f.write("Appending more")`
- You can open the file to confirm the output, or may read from it again to check

WRITING TO A FILE

- The argument of ***write()*** method must be a string
- Make sure it is casted into string first before writing to the file
- Example:
 - num1 = 23
with open("test.txt") as f:
 f.write(str(num1))
- What if you want to write multiple numbers or numbers in between text?

WRITING TO A FILE

- You must define a string separately by concatenation, as before and then use that string variable in write() method
 - Example:
 - `num1 = 24`
`str1 = "I have spotted " + str(num1) + " camels!\n"`
`f.write(str1)`